

How we used generative AI

The competition asks which AI tools we used and for what, the key prompts, and at least one place the AI was wrong and how we caught it. Everything below can be checked in the repository.

Our approach to AI

We designed the system around one principle: **AI reads, but it never decides.**

- **Our team** chose the agency (NC DHHS) and defined the user and the five match criteria from the DHHS strategic plan.
- We designed the layered pipeline, deciding that facts are handled by plain rules, that AI is used only where text has to be read, and that a published formula makes the final ranking.
- We chose to favor recall over precision, so good grants aren't lost early.
- We set the scoring weights and the penalty rules, including the separate treatment of "apply anytime" and first-come deadlines.
- We hand-labeled grants for validation.

The models below are tools inside that design.

Tools and what each did

Tool	Where	What it did	What it did not do
Jev (TypeSafe, typesafe-ai/jev, via Vercel AI Gateway)	Layer 2, src/semantic_triage.py	Screened 1,303 grants by answering our 4 multiple-choice questions for each (relevance, strategic goal, applicant role, owning division), with a probability for every answer	Never ranked grants. It only decided which went to in-depth review, and set a grant aside only above our confidence thresholds (85–95%)
DeepSeek V4 Pro (deepseek/deepseek-v4-pro, via Vercel AI Gateway)	Layer 3, src/deep_analysis.py	Filled in our structured review for 283 grants (strategic fit, ability to run it, role, division, risks, a one-line rationale), backing each claim with a verbatim quote	Never ranked. Our Python code verifies its quotes, and our fixed formula (src/scoring.py) ranks
Muse Spark 1.3 contributor (Meta, via Vercel AI Gateway)	Layer 3, first attempt	Reviewed the first 62 grants	Was blocked by the provider partway through (see below). Its 62 answers are kept only as a cross-model check

Claude Code (Anthropic)	Development	Used as a coding and writing assistant: implementing and debugging the code for the pipeline we designed, writing tests, running data checks, and helping draft documentation and slides	Made no grant judgments and set none of the criteria or weights. Every number in our results comes from the code and data in the repository
--------------------------------	-------------	--	---

No AI is used for cleaning the data, deadlines, award amounts, eligibility hard filters, the final ranking formula, or the USAspending historical-award check. Those are plain Python rules (`src/clean_data.py` , `src/deterministic.py` , `src/scoring.py` , `src/award_history.py`).

Total AI cost of a full run: **\$3.36**. Every model answer is saved in `data/cache/` , so the results reproduce without any API key.

Key prompts

Our instructions to the two pipeline models are stored as files, not buried in code.

Jev's questions (`prompts/jev_triage.json`). The one that matters most asks what role a state health agency could realistically play. We told it explicitly that "state governments" appearing in a long list of eligible applicant types does **not** make the state the intended applicant, because research grants often list every type. It chooses one of:

- **lead**: the state or its health department is an intended lead applicant
- **partner**: someone else usually leads, but the state is a natural partner
- **atypical**: technically eligible, but built for researchers, universities or small businesses
- **ineligible**: state agencies are excluded
- **unclear**

DeepSeek's instructions (`prompts/deep_analysis.txt`), in summary:

- Never invent facts. Every quote must be copied exactly from the grant, because our code checks each one and flags any that aren't found.
- Don't rank grants, and don't judge deadlines or award size; Python handles those.
- Return one structured answer: strategic fit (0–10) with the matching plan objective and an evidence quote; DHHS's role with an eligibility quote; owning division; ability to run it (0–10); restrictions; main risk; and a rationale of 30 words or fewer for a budget director.

Where the AI was wrong, and how we caught it

1. **The review model said DHHS would lead NIH research grants.** DeepSeek labeled **73** research or training grants (such as R01s and clinical trials) as DHHS-led, because "state governments" appears on their eligibility lists. **7 of our first top 10** were NIH research awards.
 - *Caught by*: reading our top 10, and comparing the two models. Jev labeled **none** of those grants as DHHS-led, and our disagreement log flagged all of them.

- *Fix*: we added an objective rule (the research activity code in the title, such as "R01") that moves research grants to a separate "pass to a university partner" list.
- 2. **It cited a single word as evidence.** For "Partnership for Disaster Health Response System", the model claimed DHHS would lead and quoted only "*states*".
 - *Caught by*: our quote check rejects quotes too short to be evidence, so the role was automatically downgraded to "unclear".
- 3. **An AI provider blocked public-health text.** Meta's Muse Spark restricted our account for "repeated policy violations" after 62 grants. Our only inputs were public grant announcements, but DHHS grants discuss HIV, STIs, sexual violence and overdoses.
 - *Caught by*: Layer 3 stalled, and a direct test returned the provider's block message.
 - *Fix*: we tested three replacement models on the 8 most sensitive grants and chose DeepSeek V4 Pro (8/8 valid answers, 8/8 quotes verified, 100% agreement with Muse on role and division).
- 4. **It ranked administrative notices as opportunities.** The model rated CDC placeholder records titled only "RFA-TS-18-000" ("Submit application as necessary") as plausible fits.
 - *Caught by*: reading our top 10.
 - *Fix*: a Layer 1 rule now removes these 17 award-transfer notices.

Full details, including non-AI failures and a case where two of our checks disagreed: `docs/failure_modes.md` (F1–F10) and `data/results/ai_error_log.csv`.